

Презентация по лабораторной работе 8

Имитационное моделирование

Нджову Нелиа

Содержание I

Цель работы

Цель этой лабораторной работы - Изучить дискретно-событийный подход к имитационному моделированию на примере классической модели распространения инфекции SIR. Реализовать стохастическую дискретно-событийную модель в виде программного комплекса на языке Julia. Провести анализ влияния параметров, сравнить со стохастической и детерминированной версиями, оценить производительность и модифицировать модель.

1 Структура программы

Задание

- 1 Структура программы
- 2 Скрипт запуска

Задание

- 1 Структура программы
- 2 Скрипт запуска
- 3 Анализ чувствительности к параметрам

Задание

- 1 Структура программы
- 2 Скрипт запуска
- 3 Анализ чувствительности к параметрам
- 4 Детерминированная длительность болезни

- 1 Структура программы
- 2 Скрипт запуска
- 3 Анализ чувствительности к параметрам
- 4 Детерминированная длительность болезни
- 5 Оценка производительности

Задание

- 1 Структура программы
- 2 Скрипт запуска
- 3 Анализ чувствительности к параметрам
- 4 Детерминированная длительность болезни
- 5 Оценка производительности
- 6 Сохранение результатов в CSV

Задание

- 1 Структура программы
- 2 Скрипт запуска
- 3 Анализ чувствительности к параметрам
- 4 Детерминированная длительность болезни
- 5 Оценка производительности
- 6 Сохранение результатов в CSV
- 7 Добавление демографических событий

Задание

- 1 Структура программы
- 2 Скрипт запуска
- 3 Анализ чувствительности к параметрам
- 4 Детерминированная длительность болезни
- 5 Оценка производительности
- 6 Сохранение результатов в CSV
- 7 Добавление демографических событий
- 8 Вакцинация

Задание

- 1 Структура программы
- 2 Скрипт запуска
- 3 Анализ чувствительности к параметрам
- 4 Детерминированная длительность болезни
- 5 Оценка производительности
- 6 Сохранение результатов в CSV
- 7 Добавление демографических событий
- 8 Вакцинация
- 9 Модель SEIR

Модель SIR - это фундаментальная сегментарная модель в эпидемиологии, которая описывает распространение инфекционного заболевания в замкнутой популяции.

Модель делит все население (N) на три отдельные, взаимоисключающие группы:

- S (Восприимчивые): люди, которые здоровы, но могут заразиться.

Общая численность населения постоянна: $N=S+I+R$.

Модель SIR - это фундаментальная сегментарная модель в эпидемиологии, которая описывает распространение инфекционного заболевания в замкнутой популяции.

Модель делит все население (N) на три отдельные, взаимоисключающие группы:

- S (Восприимчивые): люди, которые здоровы, но могут заразиться.
- I (Инфекционные): люди, которые в настоящее время больны и могут передать болезнь восприимчивым людям.

Общая численность населения постоянна: $N=S+I+R$.

Модель SIR - это фундаментальная сегментарная модель в эпидемиологии, которая описывает распространение инфекционного заболевания в замкнутой популяции.

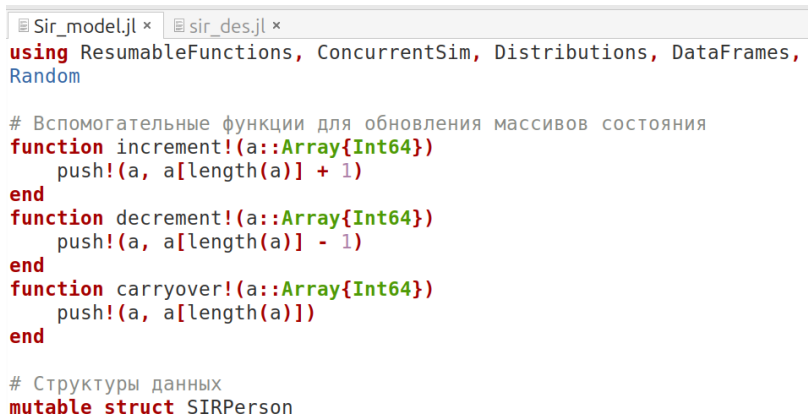
Модель делит все население (N) на три отдельные, взаимоисключающие группы:

- S (Восприимчивые): люди, которые здоровы, но могут заразиться.
- I (Инфекционные): люди, которые в настоящее время больны и могут передать болезнь восприимчивым людям.
- R (выздоровевшие): люди, которые выздоровели (и считаются невосприимчивыми) или умерли. Они больше не участвуют в процессе передачи.

Общая численность населения постоянна: $N=S+I+R$.

1. Структура программы

Я создала файл `src/Sir_model.jl` и скопировала заданный скрипт в него. Скрипт реализует ядро модели SIR с дискретными событиями. Он определяет структуры данных, логику поведения агентов и функции управления симуляцией. Я запускала код и все хорошо сработал.(рис.1)



```
using ResumableFunctions, ConcurrentSim, Distributions, DataFrames,
Random

# Вспомогательные функции для обновления массивов состояния
function increment!(a::Array{Int64})
    push!(a, a[length(a)] + 1)
end
function decrement!(a::Array{Int64})
    push!(a, a[length(a)] - 1)
end
function carryover!(a::Array{Int64})
    push!(a, a[length(a)])
end

# Структуры данных
mutable struct SIRPerson
```


2. Скрипт запуска

Я создала файл `scripts/sir_des.jl` и скопировала заданный скрипт в него. Этот скрипт загружает ядро `sir_model.jl`, устанавливает параметры моделирования, запускает моделирование SIR с дискретными событиями и генерирует график динамики S, I и R с течением времени.(рис.2)

```
# ## Модель SIR
#
# Модель создается, процессы активируются, и запуск выполняется.
# Результат преобразуется во фрейм данных. График с кривыми S, I, R
# создается и сохраняется

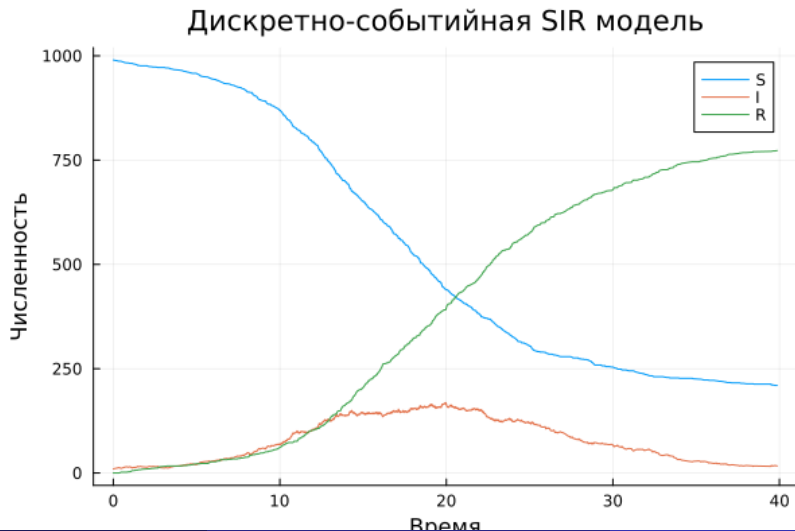
using DrWatson
@quickactivate "project"
include(scrdir("Sir_model.jl"))
using Random, StatsPlots, DataFrames, CSV

# **Разные параметры  $\beta$ **
# Фиксированные параметры
tmax = 40.0
u0 = [990, 10, 0] # S, I, R
c = 10.0
gamma = 0.25

betas = [0.05, 0.10, 0.15, 0.20, 0.25, 0.30]
results_beta = []
```

2. Скрипт запуска

Я запускала код и все хорошо сработал, получила график ниже(рис.3)



3. Анализ чувствительности к параметрам

Я создала файл `scripts/sensitivity_beta.jl`, `scripts/sensitivity_c.jl` и `scripts/sensitivity_gamma.jl`. В этих файлах, я писала код, похоже с заданный скрипт но с разными значениями β - вероятность передачи инфекции при одном контакте между заразным и восприимчивым (безразмерная величина, от 0 до 1), c - среднее число контактов (достаточно тесных для передачи инфекции) одного человека в единицу времени и γ - скорость выздоровления (доля инфицированных, выздоравливающих в единицу времени) соответственно. Я проанализировала пик заражения, время пика и конечное количество выздоровевших людей для каждого значения β

3. Анализ чувствительности к параметрам

По мере того, как β увеличивает пик заражения, число выздоровевших увеличивается, а время пика уменьшается(рис.4)

```
nelianjovu@Nebby:~/work/study/2026-1/2026-1==study--simulationmod/2026-1--study-  
-simulationmod/labs/lab08/project$ julia --project=. scripts/sensitivity_beta.jl  
 $\beta$  = 0.05 ( $R_0$  = 2.0): Пик I = 178.0, Время пика = 18.0, Финальное R = 769.0  
 $\beta$  = 0.1 ( $R_0$  = 4.0): Пик I = 379.0, Время пика = 7.6, Финальное R = 980.0  
 $\beta$  = 0.15 ( $R_0$  = 6.0): Пик I = 548.0, Время пика = 4.7, Финальное R = 995.0  
 $\beta$  = 0.2 ( $R_0$  = 8.0): Пик I = 608.0, Время пика = 4.3, Финальное R = 998.0  
 $\beta$  = 0.25 ( $R_0$  = 10.0): Пик I = 652.0, Время пика = 3.1, Финальное R = 999.0  
 $\beta$  = 0.3 ( $R_0$  = 12.0): Пик I = 729.0, Время пика = 3.0, Финальное R = 1000.0
```

Рисунок 4: Анализ разных параметрах β

3. Анализ чувствительности к параметрам

По мере того, как c увеличивается, пик заражения и число выздоровевших увеличивается, а время пика уменьшается (рис.5)

```
nelianjovu@Nebby:~/work/study/2026-1/2026-1==study--simulationmod/2026-1--study-  
-simulationmod/labs/lab08/project$ julia --project=. scripts/sensitivity_c.jl  
c = 5 ( $R_0 = 1.0$ ): Пик I = 31.0, Время пика = 36.4, Финальное R = 161.0  
c = 10 ( $R_0 = 2.0$ ): Пик I = 199.0, Время пика = 22.7, Финальное R = 767.0  
c = 15 ( $R_0 = 3.0$ ): Пик I = 342.0, Время пика = 11.8, Финальное R = 957.0  
c = 20 ( $R_0 = 4.0$ ): Пик I = 370.0, Время пика = 9.8, Финальное R = 970.0  
c = 25 ( $R_0 = 5.0$ ): Пик I = 465.0, Время пика = 6.5, Финальное R = 991.0  
c = 30 ( $R_0 = 6.0$ ): Пик I = 563.0, Время пика = 4.5, Финальное R = 995.0  
nelianjovu@Nebby:~/work/study/2026-1/2026-1==study--simulationmod/2026-1--study-  
-simulationmod/labs/lab08/project$
```

Рисунок 5: Анализ разных параметрах c

3. Анализ чувствительности к параметрам

По мере того, как γ увеличивает, пик заражения и число выздоровевших уменьшается, а время пика, сначала увеличивается а потом уменьшается(рис.6)

```
nelianjovu@Nebby:~/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab08/project$ julia --project=. scripts/sensitivity_gamma.jl
γ = 0.05 (болезнь = 20.0 дн): Пик I = 664.0, Время пика = 16.1, Финальное R = 907.0
γ = 0.1 (болезнь = 10.0 дн): Пик I = 497.0, Время пика = 15.0, Финальное R = 979.0
γ = 0.125 (болезнь = 8.0 дн): Пик I = 393.0, Время пика = 17.4, Финальное R = 975.0
γ = 0.1667 (болезнь = 6.0 дн): Пик I = 310.0, Время пика = 15.2, Финальное R = 924.0
γ = 0.2 (болезнь = 5.0 дн): Пик I = 252.0, Время пика = 14.5, Финальное R = 901.0
γ = 0.25 (болезнь = 4.0 дн): Пик I = 182.0, Время пика = 15.3, Финальное R = 817.0
γ = 0.333 (болезнь = 3.0 дн): Пик I = 102.0, Время пика = 25.1, Финальное R = 680.0
γ = 0.5 (болезнь = 2.0 дн): Пик I = 19.0, Время пика = 25.3, Финальное R = 145.0
γ = 1.0 (болезнь = 1.0 дн): Пик I = 11.0, Время пика = 0.2, Финальное R = 15.0
γ = 2.0 (болезнь = 0.5 дн): Пик I = 10.0, Время пика = 0.0, Финальное R = 11.0
```

Рисунок 6: Анализ разных параметрах γ

4. Детерминированная длительность болезни

Я заменяю экспоненциальное время восстановления на фиксированное значение $1/\gamma$ (рис.7)

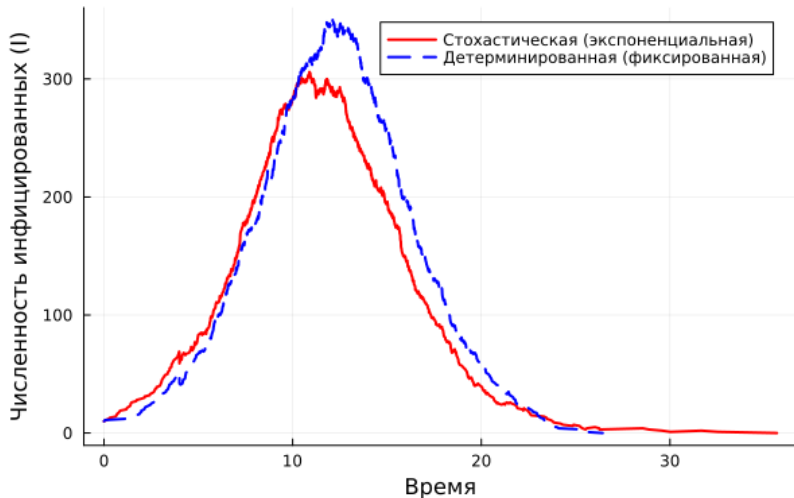
```
# Основная логика жизни индивида
@resumable function live(env::ConcurrentSim.Simulation,
individual::SIRPerson, m::SIRModel)
    while individual.status == :S
        @yield timeout(env, rand(Exponential(1/m.c)))
        alter = individual
        while alter == individual
            N = length(m.allIndividuals)
            index = rand(DiscreteUniform(1, N))
            alter = m.allIndividuals[index]
        end
        if alter.status == :I
            if rand(Uniform(0, 1)) < m.β
                individual.status = :I
                infection_update!(env, m)
            end
        end
    end
    if individual.status == :I
        @yield timeout(env, 1/m.γ)
    end
end
```

4. Детерминированная длительность болезни

Я сравниваю детерминированную версию со стохастической версией. Что касается стохастической модели, то мы можем видеть, что она немного грубовата, поскольку отражает реальность происходящего. Что касается детерминированной модели, то она немного сглажена, поскольку показывает вычислительную модель, основанную на средних значениях и уравнениях.(рис.8)

4. Детерминированная длительность болезни

Сравнение моделей SIR: динамика инфицированных



5. Оценка производительности

Используя макрос `@benchmark`, я измеряю время выполнения `sir_run` для популяции из 10 000 человек(рис.9)

```
Sir_model.jl × *sir_des.jl × *sample.jl × sir_evaluation.jl ×
using DrWatson
@quickactivate "project"
include(scrdir("Sir_model.jl"))
using Random, StatsPlots, BenchmarkTools

# Параметры модели
tmax = 40.0
u0 = [9990, 10, 0] # S, I, R
p = [0.05, 10.0, 0.25] # β, c, γ

Random.seed!(1234)

# Запуск модели
des_model = MakeSIRModel(u0, p)
activate(des_model)

bench_results = @benchmark sir_run($des_model, $tmax) samples = 10
evals = 1

# Результат
println("\nРезультаты бенчмаркинга для N=10 000:")
println(" Минимальное время: $(round(minimum(bench_result).time /
1e9, digits=4)) секунд")
```

5. Оценка производительности

Модель была протестирована на популяции из 10 000 особей в течение 40 единиц времени. Моделирование завершилось в среднем менее чем за 0,01 секунды, продемонстрировав, что реализация на основе ConcurrentSim является высокоэффективной для популяций такого размера(рис.10)

```
nelianjovu@Nebby:~/work/study/2026-1/2026-1==study--simulationmod/2026-1--study-  
-simulationmod/labs/lab08/project$ julia --project=. scripts/sir_evaluation.jl  
  
Результаты бенчмаркинга для N=10 000:  
Минимальное время: 0.0 секунд  
Медианное время: 0.0 секунд  
Среднее время: 0.0 секунд  
Максимальное время: 0.0001 секунд
```

Рисунок 10: Время выполнения программа

6. Сохранение результатов в CSV

Я добавляю автоматическое сохранение итоговой таблицы в каталог data/sims/ с уникальным именем, содержащим параметры запуска(рис.11)

```
filename = "sir_$(u0[1])_$(u0[2])_$(p[1])_$(p[2])_$(p[3]).csv"  
CSV.write(datadir("sims", filename), data_des)
```

Рисунок 11: сохранение итоговой таблицы

6. Сохранение результатов в CSV

Код сработал, и файл был сохранен(рис.12)



sir_990_
10_0.05_
10.0_0.25.
CSV

7. Добавление демографических событий

Я расширяю модель, добавляя к жизненному циклу индивидуума возможность смерти (с постоянной интенсивностью μ) и рождения новых восприимчивых особей. Смерть удаляет индивидуума из популяции, рождение добавляет нового в состояние : S.(рис 13)

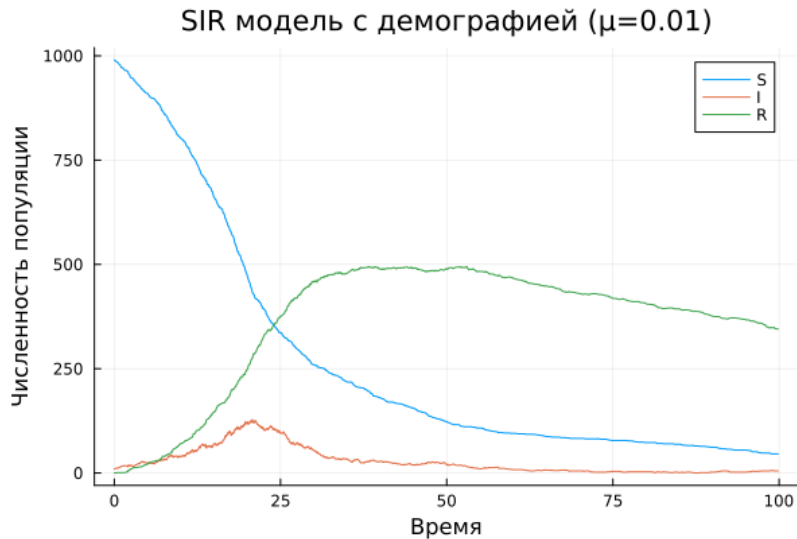
```
*sample.jl x sir_evaluation.jl x Sir_model_1.jl x sir_demographic.jl x
@yield timeout(env, Inf)
end
end
end
end

@resumable function birth_process(env::ConcurrentSim.Simulation,
m::SIRModel)
    while true
        if m.μ > 0
            @yield timeout(env, rand(Exponential(1/m.μ)))
            new_id = length(m.allIndividuals) + 1
            new_individual = SIRPerson(new_id, :S)
            push!(m.allIndividuals, new_individual)
            birth_update!(env, m)
            @process live(env, new_individual, m)
        else
            @yield timeout(env, Inf)
```

7. Добавление демографических событий

В отличие от стандартной модели SIR, в модели с демографией количество выздоровевших людей увеличивается, а затем снова начинает уменьшаться. Это связано с тем, что выздоровевшие люди могут умереть от естественных причин(рис.14)

7. Добавление демографических событий



8. Вакцинация

Я внедряю стратегию вакцинации(рис.15)

```
◀ Sir_model_1.jl x sir_demographic.jl x *Sir_model_vacc.jl x
    increment!(m.Ra)
end

function batch_vaccination_update!(sim::ConcurrentSim.Simulation,
m::SIRModel)
    push!(m.ta, ConcurrentSim.now(sim))
    current_S = count(ind.status == :S for ind in m.allIndividuals)
    current_I = count(ind.status == :I for ind in m.allIndividuals)
    current_R = count(ind.status == :R for ind in m.allIndividuals)
    push!(m.Sa, current_S)
    push!(m.Ia, current_I)
    push!(m.Ra, current_R)
end

@resumable function vaccinate(env::ConcurrentSim.Simulation,
m::SIRModel, time::Float64, fraction::Float64)
    @yield timeout(env, time)

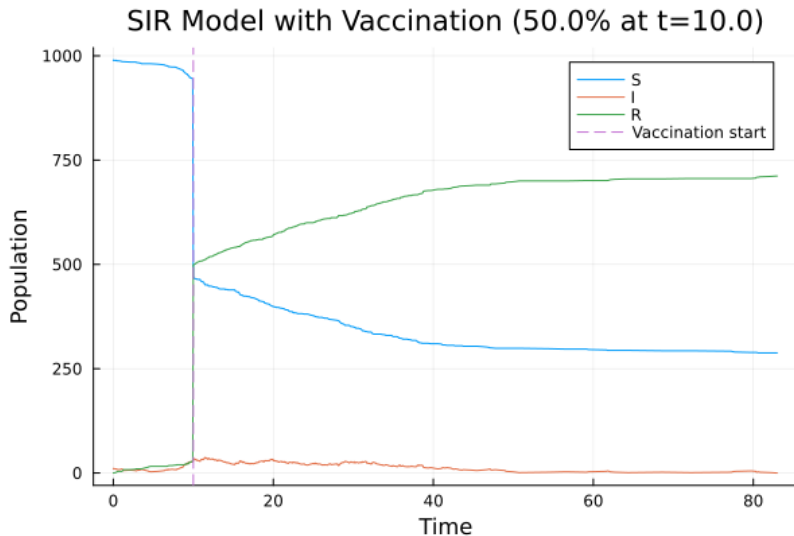
    susceptible_ids = [i for (i, ind) in enumerate(m.allIndividuals
if ind.status == :S]
    num_to_vaccinate = Int(round(length(susceptible_ids) *
fraction))

    if num_to_vaccinate > 0
        to_vaccinate = rand(m.rng, susceptible_ids,
num_to_vaccinate, replace=false)
```

8. Вакцинация

На графике мы видим, что в определенный момент времени некоторые из восприимчивых мгновенно переносят R . Для предотвращения эпидемии необходимо вакцинировать около 50% населения(рис.16)

8. Вакцинация



9. Модель SEIR

Разница между sir и $seir$ в том, что для sir это восприимчивость - заражение - выздоровление. Как только вы заразитесь, вы можете заразить других. В модели $seir$ есть дополнительный этап, который заключается в e - воздействии, в этом случае, как только вы подвергаетесь воздействию, требуется время, чтобы вы заразились и были восстановлены.

9. Модель SEIR

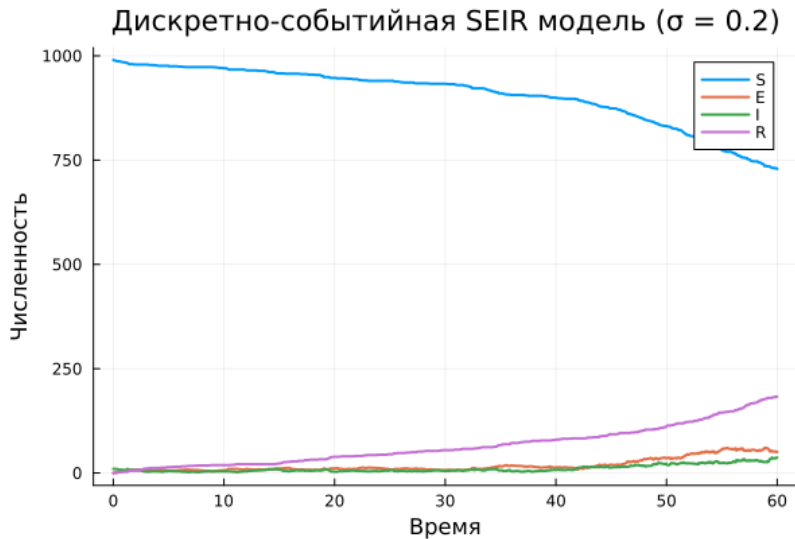
Я ввожу период ожидания (статус :E)(рис.17)

```
◀ sir_des.jl x Sir_model_vacc.jl x sir_vacc.jl x Seir.jl x seir.jl ▶  
using ResumableFunctions, ConcurrentSim, Distributions, DataFrames,  
Random  
  
# Вспомогательные функции для обновления массивов состояния  
function increment!(a::Array{Int64})  
    push!(a, a[length(a)] + 1)  
end  
function decrement!(a::Array{Int64})  
    push!(a, a[length(a)] - 1)  
end  
function carryover!(a::Array{Int64})  
    push!(a, a[length(a)])  
end  
  
# Структуры данных для SEIR  
mutable struct SEIRPerson  
    id::Int64  
    status::Symbol # :S, :E, :I, :R  
end  
  
mutable struct SEIRModel  
    sim::ConcurrentSim.Simulation  
    β::Float64  
    c::Float64
```

9. Модель SEIR

При тех же параметрах, что и в модели sir, болезнь распространяется гораздо медленнее, и в модели seir эпидемии практически отсутствуют.(рис.18)

9. Модель SEIR



В этой лабораторной работе, я изучила дискретно-событийный подход к моделированию на примере классической модели распространения инфекции SIR, как реализовать стохастическую дискретно-событийную модель в Julia и как анализировать влияние параметров, сравнивать их со стохастическими и детерминированными версиями, оценивать производительность и модифицировать систему

- 1 Кулябов Д.С. simulation-modeling-lab.pdf. 8 Реализация основных моделей в дискретно-событийном подходе. [Электронный ресурс]